APLICACIÓN DE PATRONES GOF EN EL SISTEMA POLIMARKET

PRESENTADO POR:
ALEJANDRO DE MENDOZA

POLITECNICO GRANCOLOMBIANO
MAESTRÍA ARQUITECTURA DE SOFTWARE
TEMAS AVANZADOS DE DISEÑO SOFTWARE

PRESENTADO AL PROFESOR:
ING. LINA MARIA MONTOYA SUAREZ

25 DE ABRIL
BOGOTÁ D.C.
2026

TABLA DE CONTENIDO

INTRODUCCIÓN 3

DESARROLLO DEL PROYECTO 3

Parte 1. Identificación y Justificación de los Patrones GoF 4
Patrones seleccionados 4

Parte 2. Tabla de Análisis de Patrones GoF 5

Parte 3. Diagramas UML Actualizados con Patrones GoF 6
Comparación con los diagramas de la Unidad 2 6
Tabla comparativa 9
Código PlantUML - Diagrama de componentes actualizado 9

Parte 4. Implementación de los Patrones GoF en JavaScript 11
Patrón Singleton - db.js 12
Patrón Observer - ComponenteInventario y ComponenteReabastecimiento 12
Patrón Strategy - EstrategiaCalculo 14
Patrón Factory Method - EntregaFactory 15
Patrón Facade - VentaFacade 16

Parte 5. Análisis de Modelo de Madurez - CMMI 17
Modelo de madurez seleccionado 18
Nivel de madurez del equipo 18
Fortalezas y debilidades del equipo 18
Estrategias para avanzar al Nivel 3 - Definido 19
Estrategia 1: Estandarización del proceso de desarrollo 19
Estrategia 2: Implementación de pruebas automatizadas 20
Estrategia 3: Adopción de gestión de configuración formal 20
Estrategia 4: Medición y análisis de calidad del código 20

CONCLUSIONES DEL DESARROLLO 21

BIBLIOGRAFÍA 21

AGRADECIMIENTO 21

INTRODUCCIÓN

El presente documento corresponde a la actividad sumativa de la Unidad 4 del módulo *Temas Avanzados en Diseño de Software*. Tomando como base el sistema de información PoliMarket desarrollado en la Unidad 2, cuyo diseño se estructuró mediante el uso del Lenguaje Unificado de Modelado (UML) y una arquitectura basada en componentes, el objetivo de esta entrega es fortalecer la calidad del sistema mediante la aplicación de patrones de diseño GoF (Gang of Four).

En el contexto actual del desarrollo de software, el modelado de sistemas se ha consolidado como una etapa fundamental para garantizar soluciones estructuradas, escalables y de alta calidad. A través de UML, es posible representar de manera clara y precisa la arquitectura de un sistema antes de su implementación, facilitando la comprensión de sus componentes, sus responsabilidades y las interacciones entre ellos.

Desde esta perspectiva, el enfoque orientado a objetos permite identificar las entidades principales del sistema, definir sus atributos y métodos, y establecer las relaciones existentes entre ellas. Este enfoque promueve buenas prácticas como la modularidad, el bajo acoplamiento y la alta cohesión, elementos clave para el desarrollo de aplicaciones mantenibles y extensibles, especialmente en entornos empresariales donde intervienen múltiples áreas de negocio.

El sistema PoliMarket integra diferentes áreas organizacionales como recursos humanos, ventas, bodega, proveedores y entregas, permitiendo gestionar procesos como la autorización de vendedores, el registro de ventas, la consulta de disponibilidad de productos, el reabastecimiento de inventario y la gestión de entregas. Adicionalmente, su arquitectura basada en componentes permite que cada área de negocio exponga funcionalidades específicas a través de módulos reutilizables, facilitando la separación de responsabilidades, la escalabilidad y la reutilización del software.

Asimismo, el sistema contempla la interacción mediante múltiples clientes, incluyendo una API REST para consumo web y una interfaz de línea de comandos (CLI), lo que evidencia la capacidad de reutilizar la lógica de negocio en distintas plataformas sin necesidad de duplicar código.

En continuidad con el trabajo desarrollado en la Unidad 2, en esta fase se incorpora la aplicación de cinco patrones de diseño GoF -Singleton, Observer, Strategy, Factory Method y Facade- los cuales representan soluciones probadas a problemas recurrentes del diseño orientado a objetos (Gamma et al., 1994). La inclusión de estos patrones permite mejorar la estructura del sistema, reducir el acoplamiento entre componentes y facilitar su extensibilidad y mantenibilidad.

Finalmente, la entrega incluye la identificación y justificación de los patrones seleccionados, la actualización de los diagramas de clases y componentes UML evidenciando su aplicación, una tabla de análisis por patrón con problema, solución e implementación en JavaScript, el análisis del nivel de madurez del equipo bajo el modelo CMMI, y las conclusiones del desarrollo. De esta manera, se evidencia cómo la aplicación de patrones de diseño contribuye a la construcción de sistemas más robustos, flexibles y alineados con las buenas prácticas de la ingeniería de software.

DESARROLLO DEL PROYECTO

A continuación, se presenta el desarrollo estructurado de la actividad “Reutilización de software”, orientada al análisis, diseño y mejora del sistema de información PoliMarket mediante el uso del Lenguaje Unificado de Modelado (UML), el enfoque basado en componentes y la aplicación de patrones de diseño GoF (Gang of Four). El propósito de este ejercicio es aplicar los principios fundamentales del diseño orientado a objetos y la reutilización de software, con el fin de construir una solución más robusta, flexible y mantenible, alineada con las buenas prácticas de la ingeniería de software.

El desarrollo se organiza en secciones que abordan los elementos esenciales del modelado y la arquitectura del sistema. En una primera fase, se retoman los conceptos clave identificados en la Unidad 2, los cuales son representados como clases dentro del sistema PoliMarket, justificando su inclusión a partir de los requerimientos planteados. Posteriormente, se analizan y fortalecen las relaciones entre dichas clases, incorporando principios como la asociación, la dependencia y la multiplicidad, así como mejoras derivadas de la aplicación de patrones de diseño que optimizan la estructura del sistema.

De manera complementaria, se identifican los componentes de software asociados a cada área de negocio -recursos humanos, ventas, bodega, proveedores y entregas- definiendo las funcionalidades que cada uno expone. Este enfoque permite evidenciar la reutilización de software mediante la construcción de módulos independientes, así como la optimización de su interacción a través de patrones estructurales y de comportamiento que reducen el acoplamiento y favorecen la cohesión.

Asimismo, el modelo contempla la interacción del sistema a través de múltiples clientes, específicamente una API REST y una interfaz de línea de comandos (CLI), lo que permite desacoplar la lógica del negocio de los mecanismos de acceso al sistema y facilita la reutilización de los componentes desarrollados. En esta fase, se incorpora además una capa de abstracción mediante el uso de patrones como Facade, la cual simplifica la interacción entre los clientes y los subsistemas internos, ocultando la complejidad de su coordinación.

En las secciones siguientes se presentan los patrones de diseño seleccionados y su justificación, la actualización de los diagramas de clases y componentes UML evidenciando su aplicación, así como la documentación de las funcionalidades en las que dichos patrones aportan valor. Finalmente, se expone la implementación conceptual de los patrones mediante fragmentos de código en JavaScript, junto con el análisis del modelo de madurez del equipo bajo CMMI y las conclusiones del desarrollo, demostrando cómo la aplicación de patrones de diseño contribuye a la construcción de software de mayor calidad, más escalable y adaptable a futuros cambios.

Parte 1. Identificación y Justificación de los Patrones GoF

En esta sección se presentan los cinco patrones de diseño GoF (Gang of Four) seleccionados para su implementación en el sistema PoliMarket, junto con la justificación de su elección a partir del análisis de los problemas identificados en la arquitectura desarrollada en la Unidad 2. La incorporación de estos patrones responde a la necesidad de mejorar aspectos clave del diseño, tales como el acoplamiento entre componentes, la flexibilidad ante cambios y la mantenibilidad del sistema.

La selección realizada abarca patrones pertenecientes a las tres categorías definidas en el catálogo GoF: creacionales, estructurales y de comportamiento, lo que permite abordar de manera integral diferentes tipos de problemáticas dentro del diseño orientado a objetos. Cada patrón ha sido elegido estratégicamente en función de su capacidad para resolver situaciones específicas presentes en el sistema, optimizando tanto la organización interna de los componentes como la interacción entre ellos.

De esta manera, la aplicación de estos patrones no solo fortalece la arquitectura del sistema PoliMarket, sino que también permite evidenciar el uso de soluciones de diseño probadas, contribuyendo a la construcción de un software más robusto, escalable y alineado con las buenas prácticas de la ingeniería de software.

Patrones seleccionados

A partir del análisis de la arquitectura del sistema PoliMarket, se seleccionaron cinco patrones de diseño GoF, los cuales permiten abordar problemáticas específicas relacionadas con la gestión de instancias, la comunicación entre componentes, la flexibilidad en los algoritmos de negocio y la simplificación de la interacción del sistema. A continuación, se describen los patrones seleccionados y su propósito dentro del contexto del sistema:

- Singleton (Creacional): Garantiza la existencia de una única instancia de la base de datos en memoria (db.js), asegurando que todos los componentes compartan el mismo estado. Esto evita inconsistencias en la información y centraliza el acceso a los datos del sistema.
- Observer (Comportamiento): Permite que el ComponenteInventario notifique automáticamente a otros componentes interesados, como ComponenteReabastecimiento, cuando el nivel de stock de un producto desciende por debajo de un umbral definido. Este patrón elimina el acoplamiento directo entre los componentes, facilitando la extensibilidad del sistema.
- Strategy (Comportamiento): Facilita la definición e intercambio de diferentes algoritmos para el cálculo del total de una venta, como el cálculo estándar o la aplicación de descuentos por volumen. De esta manera, se evita modificar el ComponenteRegistroVentas cada vez que se requiera incorporar una nueva lógica de cálculo, promoviendo el principio de abierto/cerrado.
- Factory Method (Creacional): Centraliza la creación de objetos relacionados con el proceso de entregas, permitiendo generar distintos tipos de entrega (por ejemplo, domicilio o express) a través

de una fábrica (EntregaFactory). Esto desacopla la lógica de instanciación de las clases concretas y facilita la incorporación de nuevos tipos de entrega en el futuro.

- Facade (Estructural): Proporciona una interfaz simplificada para el proceso de registro de ventas mediante la clase VentaFacade, la cual encapsula la coordinación entre múltiples componentes del sistema, como ComponenteAutorizacion, ComponenteInventario y las estrategias de cálculo. Esto reduce la complejidad para los clientes y mejora la usabilidad del sistema.

Parte 2. Tabla de Análisis de Patrones GoF

En esta sección se presenta la Tabla 1, en la cual se realiza un análisis detallado de cada uno de los patrones de diseño GoF seleccionados para el sistema PoliMarket. Para cada patrón, se describe el problema identificado en la arquitectura del sistema, la justificación de su aplicación como solución, así como la forma en que puede ser implementado dentro del contexto propuesto.

Adicionalmente, se incluye el método o mecanismo clave mediante el cual cada patrón contribuye a mejorar la estructura del sistema, evidenciando su impacto en aspectos como la reducción del acoplamiento, la mejora de la cohesión y la flexibilidad del diseño. Este análisis permite comprender de manera integral cómo la aplicación de patrones de diseño aporta valor al sistema, facilitando su evolución y adaptación a nuevos requerimientos.

Patrón GoF	Categoría	Problema identificado	¿Por qué ayuda?	Como implementar	Método clave
Singleton	Creacional	db.js necesita una sola instancia compartida de la BD en memoria. Sin Singleton, cada componente crearía su propio estado.	Garantiza instancia única; evita datos inconsistentes entre componentes.	Clase DB con constructor privado e instancia estática.	getInstance()
Observer	Comportamiento	Cuando el stock cae, ComponenteInventario debe notificar automáticamente a ComponenteReabastecimiento sin acoplamiento directo.	Desacopla emisor y receptor; permite agregar nuevos observadores sin modificar lógica.	ComponenteInventario como Subject; ComponenteReabastecimiento como Observer.	notificarObservadores()
Strategy	Comportamiento	El cálculo del total de una venta varía: precio estándar, descuento por volumen o precio especial por cliente.	Permite intercambiar algoritmos de cálculo sin modificar ComponenteRegistroVentas.	Interfaz EstrategiaCalculo; implementaciones: EstandarStrategy, DescuentoStrategy.	calcularTotal()
Factory Method	Creacional	Las entregas pueden ser de tipo Domicilio, Punto de Recogida o Express, y cada	Centraliza la creación de objetos; el cliente no necesita conocer	Clase EntregaFactory con método crearEntrega(tipo).	crearEntrega(tipo)

		una tiene lógica de creación diferente.	la clase concreta.		
Facade	Estructural	ComponenteRegistroVentas debe coordinar Autorizacion + Inventario + calculo + persistencia en una sola operacion. Los clientes no deben conocer estos pasos.	Simplifica la interfaz para los clientes; oculta la complejidad de la orquestacion.	VentaFacade como clase que expone solo registrarVenta() hacia los clientes.	registrarVenta()

Parte 3. Diagramas UML Actualizados con Patrones GoF

En esta sección se presentan los diagramas UML del sistema PoliMarket actualizados, en los cuales se refleja la incorporación de los cinco patrones de diseño GoF seleccionados. Estos diagramas permiten visualizar de manera estructurada las modificaciones realizadas sobre la arquitectura original, evidenciando cómo los patrones influyen en la organización de las clases, los componentes y sus interacciones.

Asimismo, se establece una comparación con los diagramas desarrollados en la Unidad 2, con el fin de identificar y analizar los cambios estructurales introducidos. Esta comparación permite evidenciar mejoras en aspectos como la reducción del acoplamiento, la incorporación de nuevos niveles de abstracción y la optimización de la comunicación entre componentes.

De esta manera, los diagramas actualizados no solo representan la estructura del sistema, sino que también demuestran el impacto de la aplicación de patrones de diseño en la evolución hacia una arquitectura más flexible, mantenible y alineada con las buenas prácticas de la ingeniería de software.

Comparación con los diagramas de la Unidad 2

En la Unidad 2 se diseñó un modelo de clases compuesto por ocho entidades de dominio principales (Vendedor, RecursosHumanos, Cliente, Producto, Venta, Bodega, Proveedor y Entrega), las cuales se relacionaban mediante asociaciones y dependencias directas. De manera complementaria, el diagrama de componentes representaba diez módulos distribuidos por área de negocio, con interacciones acopladas entre sí y sin la incorporación de mecanismos avanzados de abstracción o desacoplamiento.

En contraste, en la Unidad 4 los diagramas UML han sido actualizados para incorporar patrones de diseño GoF, lo cual introduce mejoras significativas en la estructura y organización del sistema. Estas modificaciones permiten evolucionar desde un diseño funcional hacia una arquitectura más flexible, mantenible y alineada con buenas prácticas de ingeniería de software.

Entre los principales cambios estructurales se destacan los siguientes:

- **Aplicación del patrón Singleton:**

El componente de base de datos (DB) se transforma en una instancia única gestionada mediante un constructor privado y un método de acceso estático, eliminando la posibilidad de múltiples instancias y garantizando la consistencia del estado compartido entre los componentes.

- **Implementación del patrón Observer:**

El ComponenteInventario asume el rol de Subject, incorporando una lista de observadores y el método notificarObservadores(). Por su parte, el ComponenteReabastecimiento implementa la interfaz InventarioObserver, reaccionando automáticamente ante eventos de stock bajo, lo que elimina la dependencia directa entre ambos componentes.

- **Incorporación del patrón Strategy:**

Se introduce la jerarquía EstrategiaCalculo, con implementaciones concretas como EstandarStrategy y DescuentoStrategy, permitiendo encapsular y sustituir dinámicamente los algoritmos de cálculo del total de ventas sin modificar los componentes que los utilizan.

- **Aplicación del patrón Factory Method:**

Se incorpora la clase EntregaFactory como un punto centralizado de creación de objetos, junto con la abstracción EntregaBase y sus implementaciones concretas (EntregaDomicilio y EntregaExpress). Esto desacopla la lógica de instanciación y facilita la extensión del sistema con nuevos tipos de entrega.

- **Uso del patrón Facade:**

Se introduce la clase VentaFacade, la cual encapsula la coordinación entre los subsistemas de autorización, inventario y cálculo de ventas, exponiendo una interfaz simplificada para los clientes. Esto reduce la complejidad de interacción y mejora la usabilidad del sistema.

En conjunto, estos cambios evidencian una evolución significativa del diseño inicial, pasando de una arquitectura con relaciones directas y mayor acoplamiento, a una solución basada en patrones que promueve la modularidad, la extensibilidad y la reutilización del software.

```
@startuml PoliMarket_Clases_U4
skinparam classAttributeIconSize 0
skinparam classFontSize 11
skinparam classBackgroundColor #FAFAFA
skinparam classBorderColor #555555
skinparam arrowColor #333333
skinparam shadowing false
skinparam linetype ortho

' === SINGLETON ===
class DB <<Singleton>> {
  - static instancia : DB
  + vendedores : List
  + productos : List
  + ventas : List
  + entregas : List
  - DB()
  + static getInstance() : DB
}

' === STRATEGY ===
interface EstrategiaCalculo <<Strategy>> {
  + calcular(productos: List) : double
}
class EstandarStrategy implements EstrategiaCalculo {
  + calcular(productos: List) : double
}
class DescuentoStrategy implements EstrategiaCalculo {
  + calcular(productos: List) : double
}

' === OBSERVER ===
interface InventarioObserver <<Observer>> {
  + actualizar(productoId: String, stock: int) : void
}
class ComponenteInventario <<Subject>> {
  - observadores : List<InventarioObserver>
```

```
  + agregarObservador(obs: InventarioObserver) : void
  + notificarObservadores(productoId: String, stock: int) : void
  + verificarDisponibilidad(productoId: String) : int
  + registrarEntrada(productoId: String, cantidad: int) : void
  + registrarSalida(productoId: String, cantidad: int) : void
}
class ComponenteReabastecimiento implements InventarioObserver {
  + actualizar(productoId: String, stock: int) : void
  + solicitarReabastecimiento(productoId: String) : void
}

' === FACTORY METHOD ===
abstract class EntregaBase <<FactoryMethod>> {
  # id : String
  # ventaId : String
  # estado : String
  + registrarEntrega() : void
  + confirmarEntrega() : void
}
class EntregaDomicilio extends EntregaBase {
  - direccion : String
}
class EntregaExpress extends EntregaBase {
  - tiempoMaximo : int
}
class EntregaFactory {
  + static crearEntrega(tipo: String, ventaId: String) : EntregaBase
}

' === FACADE ===
class VentaFacade <<Facade>> {
  - autorizacion : ComponenteAutorizacion
  - inventario : ComponenteInventario
  - estrategia : EstrategiaCalculo
  + registrarVenta(vendedorId: String, clienteId: String, productos: List) : Venta
}

class ComponenteAutorizacion {
  + autorizarVendedor(vendedorId: String) : boolean
  + estaAutorizado(vendedorId: String) : boolean
  + revocarAutorizacion(vendedorId: String) : void
}

class Vendedor {
  - id : String
  - nombre : String
  - autorizado : boolean
}
class Cliente {
  - id : String
  - nombre : String
  - email : String
}
class Venta {
  - id : String
  - total : double
  - estado : String
}
class Producto {
```

```
  - id : String
  - stock : int
  - precio : double
}
class Proveedor {
  - id : String
  - nombre : String
}

' RELACIONES
DB --> Vendedor
DB --> Producto
DB --> Venta

ComponenteAutorizacion --> DB
ComponenteInventario --> DB
VentaFacade --> ComponenteAutorizacion
VentaFacade --> ComponenteInventario
VentaFacade --> EstrategiaCalculo
VentaFacade --> Venta

ComponenteInventario "1" --> "0..*" InventarioObserver : notifica >
EntregaFactory ..> EntregaBase : <<crea>>
ComponenteAutorizacion "1" --> "0..*" Vendedor : autoriza >

@enduml
```

Figura 1. Diagrama de clases del sistema PoliMarket con patrones GoF - Unidad 4

Tabla comparativa

A continuación, se presenta una tabla comparativa que sintetiza las principales diferencias entre el diseño del sistema PoliMarket desarrollado en la Unidad 2 y su evolución en la Unidad 4, tras la incorporación de patrones de diseño GoF. Esta comparación permite evidenciar de manera clara y estructurada cómo las decisiones de diseño iniciales, basadas en implementaciones directas, fueron transformadas mediante la aplicación de soluciones más abstractas, flexibles y desacopladas.

En particular, se destacan cambios en aspectos clave como la creación de objetos, la lógica de cálculo, la comunicación entre componentes, el manejo de la base de datos y la interacción con los clientes del sistema. La inclusión de patrones como Factory Method, Strategy, Observer, Singleton y Facade permitió mejorar significativamente la arquitectura, reduciendo el acoplamiento, facilitando la extensibilidad y promoviendo mejores prácticas de diseño orientado a objetos.

De esta manera, la tabla no solo resume las diferencias entre ambas versiones del sistema, sino que también evidencia el impacto positivo de la aplicación de patrones de diseño en la evolución hacia una solución más robusta, mantenible y alineada con los principios de la ingeniería de software.

Aspecto	Unidad 2	Unidad 4
Creación de objetos	Directa (new)	Factory Method
Cálculo ventas	if/else	Strategy
Comunicación	Acoplada	Observer
Acceso DB	Múltiples instancias	Singleton
Interfaz cliente	Compleja	Facade

Código PlantUML - Diagrama de componentes actualizado

A continuación, se presenta el código PlantUML correspondiente al diagrama de componentes actualizado del sistema PoliMarket. Este diagrama refleja la incorporación de los patrones de diseño GoF dentro de la arquitectura basada en componentes, permitiendo visualizar cómo se reorganiza la interacción entre los clientes, los subsistemas de negocio y la infraestructura del sistema.

En esta versión, se introduce la capa Facade como intermediaria entre los clientes -API REST y CLI- y los componentes internos, simplificando el acceso a las funcionalidades principales del sistema. Asimismo, se representa la relación del patrón Observer entre ComponenteInventario y ComponenteReabastecimiento, permitiendo la notificación automática ante eventos de stock bajo.

De igual manera, se incorpora EntregaFactory como punto centralizado para la creación de distintos tipos de entregas, desacoplando la lógica de instanciación del resto de componentes. Con ello, el diagrama evidencia una arquitectura más modular, flexible y mantenible, alineada con los principios de reutilización de software y buenas prácticas de diseño orientado a objetos.

```
@startuml PoliMarket_Componentes_U4
skinparam componentStyle uml2
skinparam backgroundColor #FFFFFF
skinparam componentBackgroundColor #FAFAFA
skinparam componentBorderColor #555555
skinparam arrowColor #333333
skinparam shadowing false
skinparam linetype ortho

package "Clientes" {
  [Cliente Web (API REST)] as CW
  [Cliente CLI] as CC
}

package "Facade Layer" {
  [VentaFacade] as VF
  note right of VF : <<Facade>>
Oculta complejidad
de registro de venta
}

package "Recursos Humanos" {
  [ComponenteAutorizacion] as CA
  [ComponenteGestionVendedores] as CGV
}

package "Ventas" {
  [EstandarStrategy] as ES
  [DescuentoStrategy] as DS
  note right of ES : <<Strategy>>
Algoritmos de
calculo de total
}

package "Bodega" {
  [ComponenteInventario <<Subject>>] as CI
  [ComponenteReabastecimiento <<Observer>>] as CR
  note right of CR : <<Observer>>
Reacciona a stock bajo
}

package "Proveedores" {
  [ComponenteGestionProveedores] as CGP
  [ComponentePedidos] as CP
```

```
}

package "Entregas" {
  [EntregaFactory] as EF
  [ComponenteGestionEntregas] as CGE
  [ComponenteLogistica] as CL
  note right of EF : <<Factory Method>>
Crea tipo de entrega
segun contexto
}

package "Infraestructura" {
  [DB <<Singleton>>] as DB
  note right of DB : <<Singleton>>
Instancia unica
de base de datos
}

CW --> VF
CC --> VF
VF --> CA
VF --> CI
VF --> ES
VF --> DS
CA --> CGV
CI --> CR : <<notifica stock bajo>>
CR --> CP
CGP --> CI : <<registra entrada>>
EF --> CGE
CGE --> CL
CL --> CI : <<registra salida>>
CA --> DB
CI --> DB
CGE --> DB

@enduml
```

Figura 2. Diagrama de componentes del sistema PoliMarket con patrones GoF - Unidad 4

La utilización de estereotipos UML en los diagramas permite identificar explícitamente la aplicación de los patrones de diseño, facilitando su interpretación y alineando la representación visual con las decisiones arquitectónicas implementadas.

Parte 4. Implementación de los Patrones GoF en JavaScript

A continuación, se presenta la implementación conceptual en JavaScript, utilizando Node.js, de los cinco patrones de diseño GoF seleccionados para el sistema PoliMarket. Los fragmentos de código propuestos mantienen coherencia con la arquitectura desarrollada en la Unidad 2 y permiten evidenciar cómo cada patrón puede integrarse dentro de los componentes existentes del sistema.

Cada implementación responde a criterios básicos de diseño orientado a objetos, tales como separación de responsabilidades, bajo acoplamiento, reutilización de código y facilidad de extensión. Además, el código se encuentra documentado mediante comentarios explicativos, con el fin de facilitar la comprensión del rol que cumple cada patrón dentro de la solución propuesta.

De esta manera, esta sección no busca presentar una aplicación completamente ejecutable, sino demostrar cómo los patrones Singleton, Observer, Strategy, Factory Method y Facade pueden ser aplicados técnicamente en el contexto del sistema PoliMarket, fortaleciendo su mantenibilidad, escalabilidad y claridad arquitectónica.

Patrón Singleton - db.js

En el sistema PoliMarket, se identifica como problema que el módulo db.js es exportado e instanciado de manera directa mediante module.exports, lo que en escenarios con múltiples importaciones o contextos de ejecución puede derivar en la creación de referencias independientes. Esta situación puede generar inconsistencias en el estado de la información, afectando la integridad de los datos compartidos entre los diferentes componentes del sistema.

Para solucionar esta problemática, se aplica el patrón de diseño Singleton, el cual garantiza la existencia de una única instancia de la base de datos en memoria durante todo el ciclo de vida de la aplicación. De esta manera, todos los componentes acceden a la misma fuente de datos, asegurando coherencia, centralización y control sobre el estado del sistema.

La implementación del patrón se realiza mediante el uso de una instancia estática y un método de acceso global, evitando la creación de múltiples objetos y asegurando que cualquier solicitud de acceso retorne siempre la misma instancia compartida. Esto contribuye significativamente a la estabilidad y consistencia del sistema.

Cabe resaltar que, en entornos como Node.js, el sistema de módulos ya maneja un mecanismo de caché mediante require(), lo que en la práctica puede comportarse como una instancia única. Sin embargo, la implementación explícita del patrón Singleton se realiza con el fin de mantener consistencia conceptual en el diseño y evidenciar de manera formal su aplicación dentro de la arquitectura del sistema.

```
// db.js - Patrón Singleton
class DB {
  constructor() {
    if (DB._instancia) return DB._instancia;
    // Estado único compartido por todos los componentes
    this.vendedores = [
      { id: "V1", nombre: "Carlos Ruiz", email: "c.ruiz@polimarket.com", autorizado: false },
      { id: "V2", nombre: "Ana Torres",  email: "a.torres@polimarket.com", autorizado: false }
    ];
    this.productos = [
      { id: "P1", nombre: "Laptop", precio: 2500000, stock: 10 },
      { id: "P2", nombre: "Monitor",precio: 800000,  stock: 5  }
    ];
    this.ventas   = [];
    this.entregas = [];
    DB._instancia = this;
  }
  // Punto de acceso global a la instancia única
  static getInstance() {
    if (!DB._instancia) new DB();
    return DB._instancia;
  }
}
DB._instancia = null;
module.exports = DB.getInstance();
```

Patrón Observer - ComponenteInventario y ComponenteReabastecimiento

En el sistema PoliMarket se identifica como problemática que los componentes ComponenteInventario y ComponenteReabastecimiento se encuentran acoplados de manera directa, ya que el primero invoca explícitamente al segundo cuando se presenta una disminución en el nivel de stock. Este enfoque genera una dependencia rígida entre ambos componentes, dificultando la extensibilidad del sistema y limitando la incorporación de nuevos comportamientos asociados a eventos de inventario.

Para resolver esta situación, se aplica el patrón de diseño Observer, el cual permite desacoplar el emisor de eventos (Subject) de los componentes que reaccionan a dichos eventos (Observers). En este caso, ComponenteInventario asume el rol de sujeto, manteniendo una lista de observadores registrados, mientras que

ComponenteReabastecimiento implementa la interfaz de observador para responder ante cambios en el estado del inventario.

De esta manera, cuando el nivel de stock de un producto desciende por debajo de un umbral definido, el componente de inventario notifica automáticamente a todos los observadores registrados mediante un mecanismo de publicación de eventos, sin necesidad de conocer su implementación concreta. Esto no solo elimina el acoplamiento directo, sino que también permite incorporar nuevos observadores en el futuro sin modificar la lógica del componente principal, mejorando la flexibilidad y escalabilidad del sistema.

```
// ComponenteInventario.js - Patrón Observer (Subject)
const db = require('../db');
const STOCK_MINIMO = 3;

class ComponenteInventario {
  constructor() {
    this._observadores = []; // lista de observers registrados
  }
  // Registro de observadores
  agregarObservador(observer) {
    this._observadores.push(observer);
  }
  // Notificación a todos los observers
  notificarObservadores(productoId, stock) {
    this._observadores.forEach(obs => obs.actualizar(productoId, stock));
  }
  registrarSalida(productoId, cantidad) {
    const producto = db.productos.find(p => p.id === productoId);
    if (!producto) return { ok: false, mensaje: 'Producto no encontrado.' };
    producto.stock -= cantidad;
    // Dispara notificación si el stock baja del mínimo
    if (producto.stock <= STOCK_MINIMO) {
      this.notificarObservadores(productoId, producto.stock);
    }
    return { ok: true, stockActual: producto.stock };
  }
  verificarDisponibilidad(productoId) {
    const producto = db.productos.find(p => p.id === productoId);
    if (!producto) return { ok: false };
    return { ok: producto.stock > 0, stock: producto.stock };
  }
}
module.exports = new ComponenteInventario();

// ComponenteReabastecimiento.js - Patrón Observer (Observer concreto)
const db = require('../db');
class ComponenteReabastecimiento {
  // Método requerido por la interfaz Observer
  actualizar(productoId, stock) {
    console.log(`[Observer] Stock bajo detectado: ${productoId} = ${stock} unidades`);
    this.solicitarReabastecimiento(productoId, 'PROV-1', 20);
  }
  solicitarReabastecimiento(productoId, proveedorId, cantidad) {
    const pedido = { id: `PED-${Date.now()}`, productoId,
                     proveedorId, cantidad, estado: 'solicitado' };
    db.pedidos = db.pedidos || [];
    db.pedidos.push(pedido);
    return { ok: true, pedido };
  }
```

```
}
module.exports = new ComponenteReabastecimiento();
```

Patrón Strategy - EstrategiaCalculo

En el sistema PoliMarket se identifica como problemática que el cálculo del total de una venta se encuentra incorporado directamente dentro del método registrarVenta(), utilizando estructuras condicionales para determinar si se aplica un precio estándar, un descuento por volumen u otra regla comercial. Este enfoque genera una dependencia directa entre el componente de ventas y las reglas de cálculo, dificultando el mantenimiento del código cuando se requieren nuevas estrategias de negocio.

Para solucionar esta situación, se aplica el patrón de diseño Strategy, el cual permite encapsular cada algoritmo de cálculo en una clase independiente. De esta manera, las reglas para calcular el total de una venta dejan de estar mezcladas dentro del componente principal y pasan a representarse como estrategias intercambiables.

En este caso, se propone una interfaz o abstracción denominada EstrategiaCalculo, junto con implementaciones concretas como EstandarStrategy y DescuentoStrategy. Esto permite que el ComponenteRegistroVentas utilice una estrategia específica sin conocer los detalles internos del algoritmo aplicado.

Con esta solución, si en el futuro se requiere incorporar nuevas reglas, como descuentos por cliente frecuente, promociones temporales o impuestos diferenciados, bastará con crear una nueva estrategia sin modificar la lógica principal del componente de ventas. Esto favorece el principio de abierto/cerrado y mejora la flexibilidad, extensibilidad y mantenibilidad del sistema.

Es importante destacar que la aplicación del patrón Strategy no solo mejora la flexibilidad del sistema, sino que también permite desacoplar completamente la lógica de negocio de las reglas de cálculo, facilitando su evolución independiente. Este enfoque es especialmente útil en contextos empresariales donde las políticas comerciales cambian con frecuencia.

```
// strategies/EstrategiaCalculo.js - Patrón Strategy
// Estrategia estándar: suma precio * cantidad por producto
class EstandarStrategy {
  calcular(productos) {
    return productos.reduce((total, item) => {
      return total + (item.precio * (item.cantidad || 1));
    }, 0);
  }
}

// Estrategia con descuento: aplica 10% si hay más de 3 productos distintos
class DescuentoStrategy {
  calcular(productos) {
    const base = productos.reduce((t, item) => t + (item.precio * (item.cantidad || 1)), 0);
    const descuento = productos.length > 3 ? 0.10 : 0;
    return base * (1 - descuento);
  }
}

module.exports = { EstandarStrategy, DescuentoStrategy };

// Uso en ComponenteRegistroVentas.js
const { EstandarStrategy } = require('./strategies/EstrategiaCalculo');

class ComponenteRegistroVentas {
  constructor(estrategia = new EstandarStrategy()) {
    // La estrategia se inyecta: permite cambiarla sin modificar esta clase
    this.estrategia = estrategia;
  }
```

```
  // Permite cambiar la estrategia en tiempo de ejecucion
  setEstrategia(estrategia) {
    this.estrategia = estrategia;
  }
  calcularTotal(productos) {
    return this.estrategia.calcular(productos);
  }
}
module.exports = ComponenteRegistroVentas;
```

Patrón Factory Method - EntregaFactory

En el sistema PoliMarket se identifica como problemática que la creación de objetos de tipo Entrega dentro del ComponenteGestionEntregas se realiza mediante instanciación directa utilizando la palabra clave new, dependiendo del tipo de entrega requerido (por ejemplo, domicilio o express). Este enfoque genera un alto acoplamiento con las clases concretas, dificultando la incorporación de nuevos tipos de entrega y afectando la mantenibilidad del sistema.

Para abordar esta situación, se aplica el patrón de diseño Factory Method, el cual permite centralizar la creación de objetos en una clase especializada, delegando la lógica de instanciación a un método de fábrica. En este caso, se introduce la clase EntregaFactory, la cual expone el método crearEntrega(tipo) encargado de generar la instancia correspondiente según el tipo solicitado.

De esta manera, el ComponenteGestionEntregas deja de depender de las clases concretas y pasa a interactuar únicamente con la fábrica, lo que reduce el acoplamiento y mejora la flexibilidad del diseño. Además, esta solución facilita la extensión del sistema, ya que la incorporación de nuevos tipos de entrega solo requiere modificar o ampliar la fábrica, sin afectar el resto de los componentes.

En consecuencia, la aplicación de este patrón contribuye a una mejor organización del código, promueve la reutilización y permite que el sistema evolucione de manera más controlada y escalable.

```
// EntregaFactory.js - Patrón Factory Method
class EntregaDomicilio {
  constructor(ventaId, direccion) {
    this.id = `ENT-${Date.now()}`;
    this.ventaId = ventaId;
    this.tipo = 'domicilio';
    this.direccion = direccion;
    this.estado = 'pendiente';
  }
  registrarEntrega() {
    console.log(`Entrega domicilio programada a: ${this.direccion}`);
    return { ok: true, entrega: this };
  }
}

class EntregaExpress {
  constructor(ventaId, tiempoMaximoHoras = 2) {
    this.id = `ENT-${Date.now()}`;
    this.ventaId = ventaId;
    this.tipo = 'express';
    this.tiempoMaximo = tiempoMaximoHoras;
    this.estado = 'pendiente';
  }
  registrarEntrega() {
    console.log(`Entrega express en ${this.tiempoMaximo}h programada`);
    return { ok: true, entrega: this };
  }
}
```

```
class EntregaFactory {
  // Método de fábrica: el cliente no necesita conocer las clases concretas
  static crearEntrega(tipo, ventaId, opciones = {}) {
    switch (tipo) {
      case 'domicilio':
        return new EntregaDomicilio(ventaId, opciones.direccion || 'Sin dirección');
      case 'express':
        return new EntregaExpress(ventaId, opciones.tiempoMaximo || 2);
      default:
        throw new Error(`Tipo de entrega desconocido: ${tipo}`);
    }
  }
}
module.exports = EntregaFactory;

// Uso en ComponenteGestionEntregas.js
const EntregaFactory = require('./EntregaFactory');
function registrarEntrega(ventaId, tipo, opciones) {
  const entrega = EntregaFactory.crearEntrega(tipo, ventaId, opciones);
  return entrega.registrarEntrega();
}
```

Patrón Facade - VentaFacade

En el sistema PoliMarket se identifica como problemática que los clientes del sistema, específicamente la API REST y la interfaz de línea de comandos (CLI), deben conocer y coordinar directamente varios componentes internos para registrar una venta. Entre estos se encuentran ComponenteAutorizacion, ComponenteInventario y la lógica de cálculo del total de la venta. Esta situación genera acoplamiento, duplicación de lógica y mayor complejidad en los mecanismos de acceso al sistema.

Para resolver esta problemática, se aplica el patrón de diseño Facade, el cual permite definir una interfaz simplificada que encapsula la interacción con múltiples subsistemas. En este caso, se introduce la clase VentaFacade, encargada de coordinar internamente el proceso de registro de ventas, incluyendo la validación del vendedor, la verificación de disponibilidad de productos, la aplicación de la estrategia de cálculo y el registro de la transacción.

De esta manera, los clientes ya no necesitan conocer la lógica interna ni las dependencias entre los componentes del sistema. En lugar de interactuar con varios módulos de forma independiente, realizan una única llamada a VentaFacade, lo que simplifica el uso del sistema y reduce la posibilidad de errores.

La aplicación de este patrón mejora la organización de la arquitectura, disminuye el acoplamiento entre la capa de clientes y la lógica de negocio, y facilita el mantenimiento futuro del sistema, ya que los cambios internos pueden realizarse sin afectar directamente a los clientes que consumen sus funcionalidades.

Desde una perspectiva arquitectónica, el uso del patrón Facade también contribuye a la separación de capas dentro del sistema, actuando como punto de entrada único para los clientes. Esto permite aislar cambios en los subsistemas internos sin impactar directamente a los consumidores, lo cual es una práctica alineada con principios de diseño como la inversión de dependencias.

```
// VentaFacade.js - Patrón Facade
const ComponenteAutorizacion  = require('./ComponenteAutorizacion');
const ComponenteInventario     = require('./ComponenteInventario');
const { EstandarStrategy,
        DescuentoStrategy }    = require('./strategies/EstrategiaCalculo');
const db = require('../db');

class VentaFacade {
  constructor() {
```

```
    // Coordina internamente todos los subsistemas necesarios
    this.autorizacion = ComponenteAutorizacion;
    this.inventario   = ComponenteInventario;
    // Registra el componente de reabastecimiento como observer del inventario
    const ComponenteReabastecimiento = require('./ComponenteReabastecimiento');
    this.inventario.agregarObservador(ComponenteReabastecimiento);
  }

  // Interfaz simplificada para los clientes: una sola llamada hace todo
  registrarVenta(vendedorId, clienteId, productosIds) {
    // Paso 1: Verificar autorizacion (Facade coordina subsistema RRHH)
    if (!this.autorizacion.estaAutorizado(vendedorId))
      return { ok: false, mensaje: 'Vendedor no autorizado.' };

    // Paso 2: Verificar y descontar stock (Facade coordina subsistema Bodega)
    for (const pid of productosIds) {
      const disp = this.inventario.verificarDisponibilidad(pid);
      if (!disp.ok) return { ok: false, mensaje: `Sin stock: ${pid}` };
    }

    // Paso 3: Seleccionar estrategia de calculo segun volumen
    const productos = productosIds.map(id => db.productos.find(p => p.id === id));
    const estrategia = productos.length > 3 ? new DescuentoStrategy()
                                            : new EstandarStrategy();
    const total = estrategia.calcular(productos);

    // Paso 4: Persistir la venta
    const venta = { id: `VNT-${Date.now()}`, vendedorId,
                    clienteId, productosIds, total, estado: 'pendiente' };
    db.ventas.push(venta);

    // Paso 5: Descontar inventario (puede disparar Observer)
    productosIds.forEach(pid => this.inventario.registrarSalida(pid, 1));

    return { ok: true, mensaje: 'Venta registrada.', venta };
  }
}
module.exports = new VentaFacade();
```

Parte 5. Análisis de Modelo de Madurez - CMMI

En esta sección se presenta el análisis del nivel de madurez del equipo de desarrollo utilizando como referencia el modelo Capability Maturity Model Integration (CMMI), el cual constituye un marco ampliamente reconocido para la evaluación y mejora de los procesos en la ingeniería de software. Este modelo permite identificar el grado de formalización, control y optimización de las prácticas de desarrollo implementadas en un proyecto.

El propósito de este análisis es evaluar el estado actual del equipo a partir de la evidencia generada durante el desarrollo del sistema PoliMarket, considerando aspectos relacionados con la planificación, la calidad del software, la gestión de procesos y la documentación. A partir de esta evaluación, se busca identificar tanto las fortalezas como las debilidades del equipo, así como proponer estrategias que permitan avanzar hacia niveles superiores de madurez.

De esta manera, la aplicación del modelo CMMI no solo permite contextualizar el trabajo realizado dentro de un marco de buenas prácticas, sino que también aporta una visión crítica orientada a la mejora continua, fortaleciendo la calidad del proceso de desarrollo y la capacidad del equipo para abordar proyectos de mayor complejidad.

Modelo de madurez seleccionado

El modelo de madurez seleccionado para evaluar el desempeño del equipo de desarrollo es el Capability Maturity Model Integration (CMMI), específicamente en su versión CMMI-DEV 2.0, desarrollado por el Software Engineering Institute (SEI) de la Universidad Carnegie Mellon. Este modelo proporciona un marco de referencia estructurado para la mejora de procesos en organizaciones de desarrollo de software, permitiendo evaluar el nivel de capacidad y madurez de los equipos.

CMMI establece cinco niveles de madurez progresivos: Inicial (Nivel 1), Gestionado (Nivel 2), Definido (Nivel 3), Cuantitativamente Gestionado (Nivel 4) y Optimizado (Nivel 5). Cada nivel representa un grado mayor de control, estandarización y optimización de los procesos, facilitando la evolución hacia prácticas más eficientes y predecibles en el desarrollo de software.

La selección de este modelo se fundamenta en su amplia adopción en la industria del software empresarial, así como en su capacidad para integrar prácticas de ingeniería, gestión de proyectos y aseguramiento de la calidad dentro de un mismo marco conceptual. Además, CMMI permite realizar un análisis estructurado del estado actual del equipo, identificando oportunidades de mejora y estableciendo una ruta clara hacia niveles superiores de madurez (Chrissis et al., 2011).

Nivel de madurez del equipo

A partir del análisis de las prácticas evidenciadas durante el desarrollo del sistema PoliMarket, se determina que el equipo se encuentra en el Nivel 2 - Gestionado (Managed) del modelo CMMI. En este nivel, los proyectos son planificados, ejecutados y controlados conforme a lineamientos definidos, lo que permite garantizar la repetibilidad de los procesos dentro del contexto de proyectos individuales. Asimismo, los productos de trabajo son gestionados, revisados y documentados de manera consistente (SEI, 2018).

Esta clasificación se fundamenta en que el equipo demuestra la capacidad de producir de forma sistemática artefactos de diseño tales como diagramas UML, definición de componentes y código funcional alineado con los requerimientos del sistema. De igual manera, se evidencia una adecuada estructuración de la documentación académica, lo cual refleja un nivel de organización y control en el desarrollo del proyecto.

No obstante, aún no se alcanzan características propias del Nivel 3 - Definido, ya que no se cuenta con procesos estandarizados a nivel organizacional, ni con la adopción formal de prácticas como la automatización de pruebas, la medición de métricas de calidad o la gestión avanzada de configuración. Estos aspectos representan oportunidades de mejora para evolucionar hacia un nivel superior de madurez, donde los procesos sean definidos, documentados y aplicados de manera consistente en diferentes proyectos.

Además, como se indica, el equipo presenta características que evidencian una transición progresiva hacia el Nivel 3 (Definido), especialmente en la estructuración de artefactos de diseño, la claridad en la arquitectura basada en componentes y la calidad de la documentación técnica, lo cual sugiere una evolución hacia la estandarización de procesos.

Fortalezas y debilidades del equipo

A continuación, se presenta un análisis de las principales fortalezas y debilidades identificadas en el equipo de desarrollo, a partir de la evaluación realizada bajo el modelo CMMI. Este análisis permite evidenciar los aspectos en los cuales el equipo demuestra un desempeño sólido, así como aquellos que requieren mejora para avanzar hacia niveles superiores de madurez.

Las fortalezas reflejan las capacidades consolidadas en términos de diseño, implementación y documentación del sistema PoliMarket, mientras que las debilidades evidencian oportunidades de mejora relacionadas con la estandarización de procesos, la calidad del software y la adopción de prácticas formales de ingeniería. De esta manera, la tabla presentada a continuación permite tener una visión integral del estado actual del equipo, sirviendo como base para la definición de estrategias de mejora continua.

Aspecto CMMI	Evidencia en PoliMarket	Relacion con nivel CMMI
Fortaleza 1: Diseno orientado a objetos solido	El equipo domina UML 2.5, diagramas de clases y componentes con multiplicidades correctas.	Proceso definido con artefactos de modelado como entrada obligatoria al desarrollo (Nivel 3 CMMI).
Fortaleza 2: Arquitectura basada en componentes	Los 10 componentes estan bien delimitados con responsabilidades unicas y bajo acoplamiento.	Arquitectura documentada y reutilizable es un indicador clave de madurez en gestion de proyectos (Nivel 3).
Fortaleza 3: Implementacion funcional en Node.js	Los 5 RF estan implementados, probados y consumidos desde 2 clientes distintos.	Pruebas y validacion de productos es una practica de nivel 2 (Managed) consolidada en el equipo.
Fortaleza 4: Documentacion academica con normas APA	El equipo produce entregables con estructura formal, bibliografias y referencias correctas.	Documentacion de procesos y productos es requisito del nivel 3 (Defined) en CMMI.
Debilidad 1: Ausencia de pruebas automatizadas	No existen pruebas unitarias ni de integracion (Jest, Mocha) para los componentes.	Implementar suite de pruebas automatizadas con cobertura minima del 80%.
Debilidad 2: Gestion de configuracion informal	No se evidencia uso de ramas, versionado semantico ni politicas de merge en el proyecto.	Adoptar GitFlow con ramas feature/develop/main y versionado semantico (SEMVER).
Debilidad 3: Sin metricas de calidad del codigo	No se miden indicadores como complejidad ciclomatica, deuda tecnica ni cobertura.	Integrar SonarQube o ESLint con reglas de calidad definidas y umbrales de aceptacion.
Debilidad 4: Base de datos en memoria sin persistencia real	db.js es un mock en memoria; no hay integracion con base de datos real ni manejo de transacciones.	Migrar a una base de datos real (MongoDB o PostgreSQL) con repositorios formales.

Estrategias para avanzar al Nivel 3 - Definido

Con el propósito de avanzar del Nivel 2 - Gestionado al Nivel 3 - Definido dentro del modelo CMMI, se proponen a continuación una serie de estrategias orientadas a fortalecer la estandarización, formalización y mejora continua de los procesos del equipo de desarrollo.

Estas estrategias buscan consolidar prácticas de ingeniería de software más estructuradas, promoviendo la definición de procesos organizacionales, la mejora de la calidad del producto y la adopción de herramientas que permitan un mayor control y seguimiento del desarrollo. Su implementación permitirá no solo optimizar el desempeño del equipo, sino también garantizar la consistencia en la ejecución de proyectos futuros.

A continuación, se describen las principales estrategias que el equipo debería adoptar para alcanzar un nivel superior de madurez.

Estrategia 1: Estandarización del proceso de desarrollo

La primera estrategia consiste en la formalización y estandarización del proceso de desarrollo del equipo, mediante la documentación de un marco de trabajo común que pueda ser aplicado de manera consistente en diferentes proyectos. Esto implica definir y establecer un proceso organizacional que incluya elementos como plantillas de diseño, guías de codificación, criterios de revisión de código y listas de verificación (checklists) para la entrega de productos.

La implementación de esta estrategia permite que las actividades de desarrollo no dependan únicamente de prácticas individuales, sino que se rijan por lineamientos claros y compartidos por todo el equipo. De esta manera, se garantiza la uniformidad en la calidad de los artefactos generados y se facilita la transferencia de conocimiento entre los miembros del equipo.

Adicionalmente, la estandarización del proceso contribuye a que las prácticas adoptadas sean reproducibles en múltiples proyectos, no solo en el sistema PoliMarket, lo cual es un aspecto fundamental para alcanzar el Nivel 3 del modelo CMMI, donde los procesos deben estar definidos, documentados y aplicados de manera organizacional.

Estrategia 2: Implementación de pruebas automatizadas

La segunda estrategia consiste en la implementación de pruebas automatizadas como parte integral del proceso de desarrollo, con el fin de garantizar la calidad y confiabilidad del software. Para ello, se propone la incorporación de herramientas como Jest como framework de pruebas unitarias para los componentes del sistema PoliMarket, estableciendo como objetivo una cobertura mínima del 80% del código.

La adopción de pruebas automatizadas permite detectar errores de manera temprana, validar el correcto funcionamiento de los componentes y facilitar la evolución del sistema sin comprometer su estabilidad. Asimismo, contribuye a mejorar la mantenibilidad del código y a reducir el riesgo de fallos en futuras modificaciones.

Adicionalmente, se recomienda integrar la ejecución de pruebas dentro de un proceso automatizado mediante un pipeline básico de integración continua y entrega continua (CI/CD), utilizando herramientas como GitHub Actions o Jenkins. De esta manera, las pruebas se ejecutarán automáticamente en cada commit o pull request, garantizando un control continuo de la calidad del software y alineando al equipo con prácticas propias de niveles superiores de madurez en CMMI.

Estrategia 3: Adopción de gestión de configuración formal

La tercera estrategia se enfoca en la adopción de una gestión de configuración formal, con el objetivo de mejorar el control sobre los cambios en el código fuente y garantizar la trazabilidad de las versiones del software. Para ello, se propone la implementación de una estrategia de ramificación como GitFlow, la cual organiza el desarrollo mediante ramas principales como main, develop y ramas feature para cada nueva funcionalidad.

Esta práctica permite estructurar el trabajo del equipo, facilitar la integración de cambios y reducir conflictos en el desarrollo colaborativo. Asimismo, se recomienda establecer políticas claras de integración de código, tales como el uso obligatorio de pull requests, la revisión por al menos un miembro del equipo antes de aprobar cambios y la validación mediante pruebas automatizadas.

Adicionalmente, se plantea la adopción de estándares como el versionado semántico (Semantic Versioning - SEMVER), que permite identificar de manera clara los cambios realizados en cada versión del sistema, así como la implementación de un archivo de registro de cambios (CHANGELOG.md) que documente la evolución del software.

La aplicación de esta estrategia contribuye a mejorar la organización del desarrollo, facilita el mantenimiento del sistema y fortalece las prácticas de ingeniería del equipo, alineándolo con los requerimientos del Nivel 3 del modelo CMMI.

Estrategia 4: Medición y análisis de calidad del código

La cuarta estrategia consiste en la implementación de mecanismos formales de medición y análisis de la calidad del código, con el fin de establecer criterios objetivos que permitan evaluar y mejorar continuamente el producto de software. Para ello, se propone la integración de herramientas de análisis estático como ESLint -utilizando estándares reconocidos como las reglas de Airbnb- y plataformas de análisis de calidad como SonarQube.

Estas herramientas permiten medir indicadores clave tales como la complejidad ciclomática, la duplicación de código, la deuda técnica y la cobertura de pruebas, proporcionando una visión cuantitativa del estado del código. A partir de estos resultados, el equipo puede identificar áreas críticas, prevenir problemas futuros y tomar decisiones informadas para mejorar la mantenibilidad del sistema.

Adicionalmente, se recomienda definir umbrales mínimos de aceptación para estas métricas, los cuales deben cumplirse como condición para la entrega de cada sprint. De esta manera, se garantiza un control continuo

de la calidad del software y se promueve una cultura de mejora constante dentro del equipo, alineada con prácticas avanzadas de ingeniería y con los niveles superiores del modelo CMMI.

CONCLUSIONES DEL DESARROLLO

La aplicación de los cinco patrones de diseño GoF en el sistema PoliMarket permitió evidenciar cómo el uso de soluciones consolidadas en la ingeniería de software puede transformar una arquitectura inicialmente funcional en una más robusta, mantenible y extensible. Cada uno de los patrones implementados abordó problemáticas específicas identificadas en el diseño original, sin necesidad de alterar la lógica de negocio previamente construida, lo que demuestra su efectividad como mecanismo de mejora incremental.

En particular, el patrón Singleton garantizó la consistencia en el manejo de la base de datos en memoria, evitando múltiples instancias y posibles inconsistencias en el estado del sistema. Por su parte, el patrón Observer permitió desacoplar la comunicación entre los componentes de inventario y reabastecimiento, facilitando la incorporación de nuevos observadores sin modificar la lógica del componente principal. Asimismo, el patrón Strategy promovió la separación de los algoritmos de cálculo del total de ventas, alineándose con el principio de abierto/cerrado (Open/Closed) y mejorando la flexibilidad del sistema.

De igual manera, el patrón Factory Method centralizó la creación de objetos relacionados con las entregas, eliminando la dependencia de clases concretas y facilitando la extensión del sistema. Finalmente, el patrón Facade permitió simplificar la interacción de los clientes con el sistema, encapsulando la complejidad interna y mejorando la usabilidad de la solución.

Desde la perspectiva del modelo de madurez CMMI, se determinó que el equipo se encuentra en el Nivel 2 - Gestionado, evidenciando procesos repetibles y una adecuada producción de artefactos de calidad. No obstante, las estrategias propuestas -como la estandarización del proceso, la implementación de pruebas automatizadas, la gestión de configuración formal y la medición de la calidad del código- establecen una ruta clara para evolucionar hacia el Nivel 3 - Definido, donde los procesos organizacionales adquieren un papel central en el desarrollo.

En síntesis, esta actividad permitió comprender que los patrones de diseño GoF no constituyen un fin en sí mismos, sino herramientas que facilitan la toma de decisiones de diseño más acertadas. La integración de un modelado UML riguroso, la aplicación de patrones de diseño y el uso de marcos de evaluación como CMMI conforman una base sólida para el desarrollo de software profesional, sostenible y orientado a la mejora continua.

BIBLIOGRAFÍA

A continuación, se presenta la bibliografía utilizada como base conceptual y metodológica para el desarrollo del presente proyecto:

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). El lenguaje unificado de modelado (UML): Guía del usuario (2.ª ed.). Pearson Educación.
- Chrissis, M. B., Konrad, M., & Shrum, S. (2011). CMMI for development: Guidelines for process integration and product improvement (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Object Management Group. (2017). Unified Modeling Language (UML) Specification Version 2.5. https://www.omg.org/spec/UML/
- PlantUML. (2024). PlantUML language reference guide. https://plantuml.com/
- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería de software: Un enfoque práctico (9.ª ed.). McGraw-Hill.
- SEI - Software Engineering Institute. (2018). CMMI for development, version 2.0. Carnegie Mellon University. https://cmmiinstitute.com/
- Sommerville, I. (2016). Ingeniería de software (10.ª ed.). Pearson Educación.

AGRADECIMIENTO

Expreso mi más sincero agradecimiento a la Ingeniera Lina María Montoya Suárez por su orientación, acompañamiento y compromiso durante el desarrollo de la presente actividad. Su guía ha sido fundamental para

fortalecer la comprensión de los conceptos relacionados con el modelado de software, la reutilización de componentes y la aplicación del diseño orientado a objetos en contextos reales.

Asimismo, agradezco el espacio académico brindado, el cual permitió aplicar de manera práctica los conocimientos adquiridos, no solo en la construcción de modelos UML, sino también en la estructuración de arquitecturas basadas en componentes y en la implementación de soluciones funcionales mediante herramientas tecnológicas como Node.js.

Finalmente, valoro profundamente el acompañamiento constante y las herramientas proporcionadas a lo largo del proceso, las cuales contribuyeron significativamente al desarrollo de este trabajo y al fortalecimiento de mis competencias en el análisis, diseño e implementación de sistemas de software, desde una perspectiva estructurada, orientada a la calidad y a la reutilización.

Respetuosamente,

ALEJANDRO DE MENDOZA
Ingeniero Informático
Ingeniero de Plataformas e Integraciones